

ADT Bag

Bag = container with no positions (~ shopping cart)

- a container in which - the order of elements is not important
- the elements do not have to be unique.

=> there are no positions (no operations working (returning pos.))

ex: dictionary in Python

there will be no operations working with positions

assume that you have an empty bag.

- ADD the following elements:

1, 2, 1, 7, 4, 2, 3, 1

- print the content of the bag:

1, 2, 1, 7, 4, 2, 3, 1

1, 1, 1, 2, 2, 3, 4, 7

1, 1, 1, 2, 2, 7, 4, 3

→ random order

(order of memory) ←

Formal def:

- Domain: $B = \{b \mid b \text{ is a bag with elements of type } T \text{ elem}\}$.

- Interface: $\text{add}(b, e)$

(shortest) $\text{remove}(b, e)$

(limited capacity) $\text{size}(b)$

$\text{search}(b, e) \rightarrow$ how to search with no positions?

$\text{init}(b)$

$\text{destroy}(b)$

$\text{iterator}(b, it) \rightarrow$ result (output)

$\text{nrOfOccurrences}(b, e)$

! update does not make sense because we have no positions (for bags)

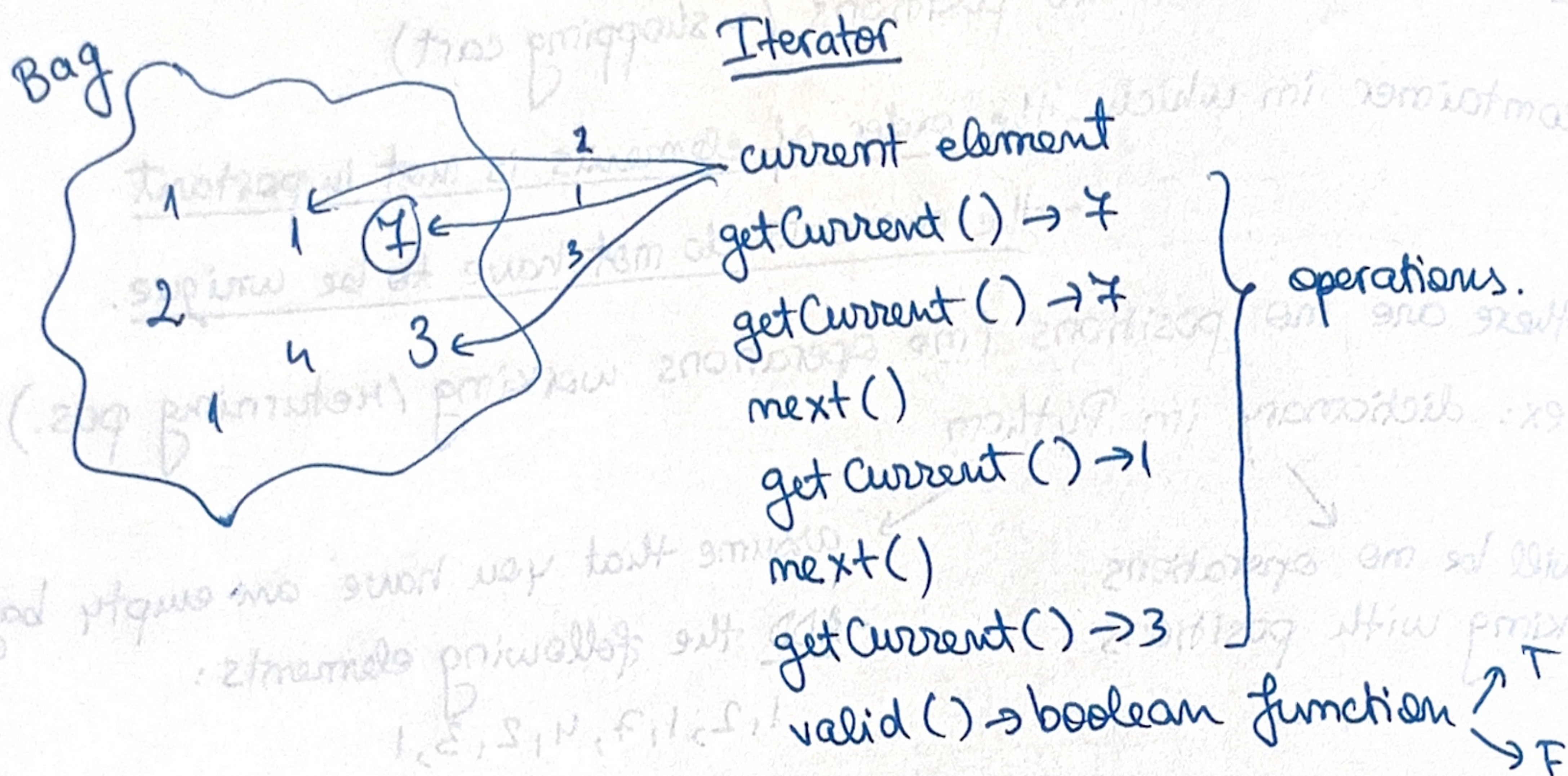
? What happens in Python if I try to remove an element that is not there?

=> we should throw an exception.

two approaches.

b' $\rightarrow$ changing the content.

! We don't have access to the elements/content.



? How to know what is input and what is output?

$\downarrow$

$\downarrow$

it

$\downarrow$

get created inside function
($\nexists$ pre, but $\exists$ post conditions).

ADT Iterator

- uni-directional
- bi-directional

- give container as a parameter.

• Domain :

$I = \{it \mid it \text{ is an iterator over a bag } b\}$

• Interface :

init(i, b) $\rightarrow$ constructor

iterator(b, it) $\rightarrow$ call the constructor

! Never pass constructor or iterator directly $\rightarrow$ use iterator function.

next $\rightarrow$ changes internally (i')

first(i) $\rightarrow$ go back to the beginning (iterator)

Representation:

Bag $\rightarrow$ more data types for representations.

- ① Use a LIST for the elements.
 $\hookrightarrow$ already a container

ex:

1	7	2	1	4	3	1	2	4
---	---	---	---	---	---	---	---	---

$\rightarrow$ current element from the iterator is the index/position of the current element.

- ② Use a list of UNIQUE ELEMENTS + FREQUENCIES.

elems:

1	7	2	4	3
0	1	2	3	4

freq:

3	1	2	2	1
---	---	---	---	---

$\rightarrow$ current element from the iterator is the current position + current frequency

Implementation:

```
class Bag:
```

```
    def __init__(self):  
        self.--bag = list()
```

```
    def add(self, e):  
        self.--bag.append(e)
```

```
    def remove(self, e):  
        if e in self.--bag:  
            self.--bag.remove(e)  
            return True  
        return False
```

```
    def search(self, e):  
        return e in self.--bag
```

```
    def size(self):  
        return len(self.--bag)
```

```
    def noOfOccurrences(self, e):  
        count = 0  
        for elem in self.--bag:  
            if elem == e:  
                count = count + 1
```

```
        return count
```



```
def iterator(self):
    return BagIterator(self)
```

class BagIterator:

```
def __init__(self, b):
    self.__bag = b
    self.__current = 0

def get_current(self):
    return self.__bag[self.__current]

def next(self):
    self.__current += 1

def valid(self):
    return self.__current < self.__bag.size()

def first(self):
    self.__current = 0
```

CONSTRUCTOR

name mangling

Correct:

```
def get_current(self):
    if self.valid():
        return self.__bag[self.__current]
    raise ValueError()

def next(self):
    if self.valid():
        self.__current += 1
    else:
        raise ValueError()
```

```
return self.__bag[self.__current]
# (not allowed in Python; uprivate)
# (name mangling -> renamed)
```

name of class.

class BagFr:

```
def __init__(self):
    self.__elems = []
    self.__fr = []
```

```
def iterator(self):
    return BagFrIterator(self)
```

```
def createIntBag():
    b = BagFr()
    b.add(1)
```

```
def printBag(b):
    it = b.iterator()
    b = createIntBag()
    printBag(b)
```

one single small change in my code.

class BagFrIterator:

```
def printBag(b):
    it = b.iterator()
    while it.valid():
        e = it.get_current()
        print(e)
        it.next()
```

knows what kind of iterator to create to match your bag.